

Module 2 FEM Implementation Note

Detailed How-To Guide for RadiationEffects_proj2

Running, testing, inspecting, modifying, and diagnosing the two-dimensional electrostatic finite-element solver

Purpose and scope. This is the operational companion to the Module 2 physics note. It documents the MATLAB finite-element method (FEM) implementation of the two-dimensional electrostatic Poisson problem on both the original rectangular verification domain and the validated Module 9 transmon substrate mesh. It also documents the new arbitrary nodal charge-field input contract and the factorized multi-configuration FEM dataset path beginning at `main_module2_batch_fem_dataset.m`. The canonical Module 2 input is now the complete nodal total-charge field ρ ; Gaussian parameters are retained only as synthetic field-generator metadata. It explains what to run, how named electrode tags are consumed, which files are called, what each returned field means, how to execute all legacy, transmon, and batch verification tests, and how to diagnose an unexpected result. The separate PINN guide remains authoritative for the neural-network path.

Fastest correct start. Open MATLAB in the repository's `matlab/` directory, run `setup_project_paths`, then run `test_module2_fem_all_2d`. After the suite passes, generate the two geometry-aware demonstrations with `main_module2_electrostatics('transmon_laplace')` and `main_module2_electrostatics('transmon_trapped_charge')`. Then generate the field-to-field surrogate pilot data with `[dataset,report] = main_module2_batch_fem_dataset()`. The batch MAT-file and validation summary are written under `matlab/outputs/module2_batch_fem_2d/`; plots remain disabled by default.

Contents

Symbols and acronyms used in this document	4
1 What the Module 2 FEM code does	5
1.1 Implemented capabilities	5
1.2 Important current limitations	6

2	Software requirements and starting location	6
2.1	Required software	6
2.2	Correct working directory	7
2.3	Path preflight check	7
3	Quick-start execution recipes	8
3.1	Default run	8
3.2	Run all six built-in FEM cases	8
3.3	Generate the 17-case Module 9 pilot dataset	8
3.4	Run one case from a system terminal	9
3.5	Run without writing plots or a MAT-file	9
4	Built-in cases and expected behavior	9
5	File map and responsibility of each component	11
6	Exact execution sequence	14
7	Batch FEM dataset contract and execution sequence	14
8	Parameter reference	16
8.1	Physical constants and material	16
8.2	Geometry and mesh	16
8.3	Canonical charge-field input and synthetic source controls	17
8.4	Boundary conditions	19
8.5	Output controls	19
9	Custom runs without editing the baseline defaults	20
9.1	Mesh-refined localized source	20
9.2	Uniform dopant-only source	20
9.3	Saving a direct-solver result deliberately	20
10	Returned data structure: what to inspect	21

11 How to probe a run numerically	22
11.1 Dimensions, ranges, and finiteness	22
11.2 Linear-system residual	22
11.3 Boundary-condition error	22
11.4 Matrix structure and conditioning	23
11.5 Probe the node nearest a physical coordinate	23
11.6 Inspect one triangle	23
12 Generated outputs and how to check them	24
13 Verification tests	25
13.1 Run the FEM-only test suite	25
13.2 What each test proves	26
13.3 Batch-test command	28
13.4 Mesh-refinement check	28
14 Debugging and interactive code tracing	29
14.1 Stop at the first MATLAB error	29
14.2 Enter the solver at its first line	29
14.3 Inspect source code and resolution	29
15 Common failure modes and diagnostic actions	30
16 Adding a new standalone case safely	31
16.1 One-off experiment	31
16.2 New named regression or demonstration case	31
17 Preparing for Module 3 and Module 5 coupling	32
18 Recommended verification ladder before using a result	32
19 Minimal handoff checklist	33
20 Next development gate	34

Symbols and acronyms used in this document

Symbol / acronym	Name	Meaning in this document
FEM	Finite-element method	Galerkin discretization using three-node linear triangular elements.
T3	Three-node triangle	Linear triangular element with one potential degree of freedom at each vertex.
Ω	Computational domain	Either the legacy rectangle $[0, L_x] \times [0, L_y]$ or Module 9 substrate $[-1.5, 1.5] \text{ mm} \times [-0.5, 0] \text{ mm}$.
ϕ	Electrostatic potential	Primary FEM unknown, stored at mesh nodes in volts.
$\mathbf{E}$	Electric field	Postprocessed as $\mathbf{E} = -\nabla\phi$, in volts per meter.
ρ	Space-charge density	Volumetric source in coulombs per cubic meter.
ϵ_{Si}	Silicon permittivity	Product of relative and vacuum permittivity, in farads per meter.
N_a	Shape function	Linear basis function for local triangle node a .
K	Stiffness matrix	Unconstrained sparse global Poisson matrix.
K_{bc}	Boundary-modified matrix	Matrix after strong Dirichlet constraints are imposed.
$\mathbf{b}$	Source vector	Assembled right-hand side before boundary modification.
Φ	Nodal potential vector	Solution of $K_{\text{bc}}\Phi = \mathbf{b}_{\text{bc}}$.
ρ	Nodal charge-field input	One total charge-density value per FEM node; canonical Module 2 input.
μ_{gen}	Synthetic-generator metadata	$[C_{\text{peak}}, x_c, z_c, \sigma_x, \sigma_z]$ for the default Gaussian pilot only.
BC	Boundary condition	Dirichlet potential or natural homogeneous Neumann field condition.

Symbol / acronym	Name	Meaning in this document
SI	International System of Units	Unit convention used by all Module 2 parameters.

1 What the Module 2 FEM code does

The solver computes the electrostatic potential generated by mobile carriers, ionized dopants, and an effective charged-defect population. The strong-form equation is

$$\nabla \cdot (\varepsilon_{\text{Si}} \nabla \phi) = -\rho, \quad \mathbf{E} = -\nabla \phi. \quad (1.1)$$

The implemented weak form is

$$\int_{\Omega} \varepsilon_{\text{Si}} \nabla w \cdot \nabla \phi \, d\Omega = \int_{\Omega} w \rho \, d\Omega, \quad (1.2)$$

with prescribed potential on selected boundaries and zero normal potential gradient on natural boundaries. The FEM approximation produces

$$K\Phi = \mathbf{b}. \quad (1.3)$$

The current path is a **steady, linear, single-material electrostatic solve**. There is no time step in Module 2 itself. Time dependence enters a future coupled calculation only because Module 3, Module 4, and Module 5 can supply a new charge state at successive coupled times.

1.1 Implemented capabilities

- Legacy structured rectangular meshes and exact reuse of the validated nonuniform Module 9 substrate mesh.
- Constant silicon permittivity throughout the domain.
- Explicit arbitrary nodal total-charge fields supplied directly to the solver.
- Composition of scalar or nodal electron, hole, ionized-dopant, and charged-defect fields into total ρ .
- Gaussian and uniform charge generators retained as synthetic verification/data-generation utilities.
- Strong constant-valued Dirichlet conditions on rectangular sides or named Module 9 tags.

- Natural homogeneous Neumann conditions on sides that are not Dirichlet.
- Sparse global assembly, direct sparse solution with MATLAB's backslash operator, element electric fields, and area-weighted nodal electric fields.
- Explicit free-node residual, fixed-voltage error, and global source/reaction balance diagnostics.
- Transmon field plots with left-electrode, right-electrode, and JJ overlays in (x, z) coordinates.
- MAT-file, PNG-figure, and text-summary output through the top-level driver.
- Deterministic 17-field pilot dataset storing complete $\rho \rightarrow \Phi$ pairs, with one stiffness assembly, one sparse Cholesky factorization, and complete-field train/validation/test splits.

1.2 Important current limitations

- The implementation does *not* assemble a nonzero Neumann boundary integral. The fields `params.bc.*.dphidn` are descriptive placeholders for the present zero-flux cases. Changing one to a nonzero value does not change the solution.
- Permittivity is supplied as one scalar. Spatially varying or tensor permittivity is not yet supported.
- Dirichlet values are scalar constants per side or named tag; a spatially varying contact profile is not accepted by the current boundary collector.
- The solver does not solve carrier statistics or trap occupancy self-consistently. It accepts an externally prepared total charge field, or prescribed component fields that are combined before the Poisson solve.
- The thin metal films remain zero-thickness boundary tags. This electrostatic solve does not resolve metal or oxide thickness as bulk elements.
- The JJ tag is a scoring/interface location, not a resolved Josephson constitutive model.
- An all-Neumann problem is singular. The solver now stops explicitly when no Dirichlet nodes are found.

2 Software requirements and starting location

2.1 Required software

The Module 2 FEM path calls core MATLAB operations only: structures, sparse matrices, backslash and sparse Cholesky solves, basic plotting, file I/O, and path management. It does not call the

Deep Learning Toolbox and does not require the Module 2 PINN files. Geant4 is also not required for the six standalone FEM cases or the batch dataset generator.

The practical requirements are:

- MATLAB with permission to read the repository and write under `matlab/outputs/`;
- sufficient memory for the requested mesh;
- a graphics-capable MATLAB installation when `makePlots=true`. The figures are created with `'Visible','off'`, so an interactive figure window is not required in the usual batch configuration.

2.2 Correct working directory

The least ambiguous starting point is the project's MATLAB directory:

```
1 cd('C:/path/to/RadiationEffects_proj2/matlab')
2 setup_project_paths
```

Replace the example path with the actual extracted repository path. Forward slashes work in MATLAB on Windows. If MATLAB was opened directly in `matlab/`, only the second line is needed.

`setup_project_paths.m` adds the following directories:

- `matlab/`;
- `matlab/cases/`;
- `matlab/tests/`;
- `matlab/pinn/`;
- all folders recursively below `matlab/src/`.

The PINN directory is added for the complete project, but it is not used by the FEM run documented here.

2.3 Path preflight check

Before solving, confirm that MATLAB resolves the intended files:

```
1 which main_module2_electrostatics -all
2 which solve_poisson_defect_space_charge_2d -all
3 which default_module2_params -all
```

Each command should identify the corresponding file inside the current project's `matlab/` tree. Multiple results can indicate that an older project copy is still on the MATLAB path. Remove the stale path or start a fresh MATLAB session before comparing results.

3 Quick-start execution recipes

3.1 Default run

Calling the driver without an argument selects the localized charged-defect case:

```
1 setup_project_paths
2 result = main_module2_electrostatics;
```

This is equivalent to

```
1 result = main_module2_electrostatics('localized_defect_charge');
```

3.2 Run all six built-in FEM cases

```
1 setup_project_paths
2
3 r0 = main_module2_electrostatics('zero_charge');
4 r1 = main_module2_electrostatics('linear_potential');
5 r2 = main_module2_electrostatics('uniform_space_charge');
6 r3 = main_module2_electrostatics('localized_defect_charge');
7 r4 = main_module2_electrostatics('transmon_laplace');
8 r5 = main_module2_electrostatics('transmon_trapped_charge');
```

The same output filenames are overwritten when the same case is rerun. Distinct built-in case names do not overwrite one another because the case name prefixes every generated file.

3.3 Generate the 17-case Module 9 pilot dataset

The batch generator is a separate user-facing driver. It does not create one result structure or one set of figures per configuration:

```
1 setup_project_paths
2 test_module2_batch_fem_dataset_2d
3 [dataset, report] = main_module2_batch_fem_dataset();
```


The default call saves one compact MAT-file and one text manifest. It creates no PNGs and opens no figures. To export only four explicitly configured audit cases, opt in deliberately:

```
1 options = default_module2_batch_fem_options_2d();
2 options.makeRepresentativePlots = true;
3 [dataset, report] = main_module2_batch_fem_dataset(options);
```

3.4 Run one case from a system terminal

MATLAB's batch interface can run the solver without first opening the desktop:

```
matlab -batch "cd('C:/path/to/RadiationEffects_proj2/matlab'); setup_project_paths;
              result = main_module2_electrostatics('localized_defect_charge');"
```

The terminal process returns a nonzero exit status if MATLAB encounters an uncaught error, which is useful for automated verification.

3.5 Run without writing plots or a MAT-file

The top-level driver constructs its own default parameter structure, so output flags cannot be passed through its function signature. For a diagnostic solve with reduced file output, call the parameter factory and solver directly:

```
1 setup_project_paths
2 params = default_module2_params('localized_defect_charge');
3 params.makePlots = false;
4 params.saveMat = false;
5 result = solve_poisson_defect_space_charge_2d(params);
```

In this direct-solver path, `makePlots` and `saveMat` are not read because plotting and saving belong to the top-level driver. The flags are still useful when the same `params` structure is later passed through a custom driver. The direct call itself writes no files unless plotting and summary functions are invoked explicitly.

4 Built-in cases and expected behavior

Case name	Configuration	Expected result and purpose
zero_charge	$\rho = 0$; left and right potentials both zero.	$\phi = 0$ and $\mathbf{E} = 0$ to numerical precision. This detects unintended sources or boundary values.
linear_potential	$\rho = 0$; left potential 0 V and right potential 1 V.	Exact Laplace solution $\phi = x/L_x$ with uniform $E_x = -1/L_x$. This checks mesh assembly, Dirichlet insertion, and field sign.
uniform_space_charge	Explicit $\rho_0 = 5.0 \times 10^{-4}$ C/m ³ ; left and right potentials zero.	Nearly one-dimensional quadratic solution $\phi = \rho_0 x(L_x - x)/(2\epsilon_{\text{Si}})$. This checks the source vector.
localized_defect_charge	Positive Gaussian effective defect population centered in the device; grounded left and right contacts.	Finite positive potential perturbation and localized positive charge. This is a physical sanity case, not an exact-solution verification.
transmon_laplace	Module 9 mesh; $\rho = 0$; 0 V on <code>left_electrode</code> and 1 mV on <code>right_electrode</code> ; all other boundaries natural.	Mixed-boundary Laplace field on the chip cross-section. This verifies named-tag constraints, electrode separation, maximum range, residual, and reaction balance.
transmon_trapped_charge	Module 9 mesh; both named electrodes grounded; positive Gaussian defect charge centered $20\mu\text{m}$ below the JJ.	Positive localized potential perturbation on the shared transmon mesh. This tests charge mapping, tagged grounding, residual, and global source/reaction balance.

The files in `matlab/cases/` are small convenience functions that return these parameter structures. The main driver does not call the case functions; it calls `default_module2_params(caseName)` directly. For example,

```

1 params = case_module2_linear_potential_2d();
2 result = solve_poisson_defect_space_charge_2d(params);

```

is a valid direct-solver route, but it bypasses the driver's automatic saving, plotting, summary writing, and console report.

5 File map and responsibility of each component

File	Responsibility
matlab/main_module2_ electrostatics.m	User-facing driver. Selects a built-in case, runs the solver, creates the output directory, saves results, makes plots, writes a summary, and prints scalar diagnostics.
matlab/main_module2_batch_fem_ dataset.m	User-facing multi-configuration driver. Generates, validates, and saves one Module 9-aware dataset; representative plots are explicit opt-in outputs.
matlab/setup_project_paths.m	Adds the driver, cases, tests, PINN directory, and all source subdirectories to the MATLAB path.
matlab/src/module2_ electrostatics/default_module2_ params.m	Defines constants, geometry mode, synthetic demonstration parameters, side/tag boundary conditions, output controls, and supported named cases.
matlab/src/module2_ electrostatics/make_module2_ charge_field_2d.m	Packages and validates the canonical arbitrary nodal total-charge field.
matlab/src/module2_ electrostatics/validate_ module2_charge_field_2d.m	Validates the field schema, nodal count, representation, units, and finite values.
matlab/src/module2_ electrostatics/compose_module2_ charge_field_2d.m	Combines scalar or nodal $p, n, N_D^+, N_A^-, C_{\text{def}}, z_{\text{def}}$ fields into total ρ .
matlab/src/module2_ electrostatics/generate_ module2_gaussian_charge_field_ 2d.m	Synthetic Gaussian/uniform field generator for controlled cases; not the canonical solver interface.
matlab/src/module2_ electrostatics/build_space_ charge_module2_2d.m	Legacy compatibility wrapper retained for the current pointwise PINN prototype and old scripts.
matlab/src/module2_ electrostatics/solve_poisson_ defect_space_charge_2d.m	Numerical orchestrator. Accepts an explicit nodal charge field, then performs assembly, boundary processing, sparse solve, and field postprocessing.

File	Responsibility
matlab/src/module2_ electrostatics/solve_module2_ nodal_charge_batch_2d.m	Generic multi-field solver that reuses one operator/-factorization for arbitrary ρ columns.
matlab/src/module2_ electrostatics/resolve_module2_ mesh_2d.m	Selects the legacy rectangle or validates and reuses a supplied Module 9 geometry; synchronizes domain and substrate permittivity metadata.
matlab/src/module2_ electrostatics/make_ rectangular_tri_mesh_2d.m	Creates nodes, counter-clockwise T3 connectivity, coordinate vectors, dimensions, and left/right/top/bottom boundary-node sets.
matlab/src/module2_ electrostatics/assemble_ poisson_fem_2d.m	Loops over triangles, forms local stiffness and source contributions, scatters them into a sparse global matrix and right-hand side, and stores element areas and centroids.
matlab/src/module2_ electrostatics/assemble_ poisson_source_rhs_2d.m	Reuses stored triangle areas to assemble only the new source vector for each batch configuration.
matlab/src/module2_ electrostatics/compute_element_ stiffness_triangle.m	Calculates signed area, shape-function gradients, and $K_e = \epsilon A_e (\nabla N)(\nabla N)^T$. Rejects non-positive element orientation.
matlab/src/module2_ electrostatics/compute_element_ source_triangle.m	Calculates the consistent T3 source vector using the triangle mass-like matrix multiplied by the three nodal charge values.
matlab/src/module2_ electrostatics/get_module2_ dirichlet_nodes.m	Collects nodes and scalar values from rectangle sides and named Module 9 tags. Rejects missing, empty, or mutually conflicting constraints.
matlab/src/module2_ electrostatics/apply_dirichlet_ bc.m	Adjusts the right-hand side for known values, zeros constrained rows and columns, and inserts unit diagonal equations.
matlab/src/module2_ electrostatics/compute_ electric_field_from_potential_ 2d.m	Computes elementwise $-\nabla\phi$ and area-weighted nodal averages.
matlab/src/module2_ electrostatics/compute_module2_ fem_diagnostics_2d.m	Computes free-node residual, Dirichlet error, constraint reactions, and discrete global source/reaction balance.

File	Responsibility
matlab/src/module2_electrostatics/plot_module2_result_2d.m	Writes potential, space-charge, and electric-field-magnitude PNG figures.
matlab/src/module2_electrostatics/write_module2_summary.m	Writes geometry, mesh, material, source, potential, field, and boundary-node scalar values to a text file.
matlab/src/module2_electrostatics/default_module2_batch_fem_options_2d.m	Defines dataset name, design, fixed electrode voltages, output controls, representative cases, and validation tolerances.
matlab/src/module2_electrostatics/make_module2_gaussian_field_pilot_design_2d.m	Defines deterministic Gaussian-generator metadata for the 17-field pilot and complete-field split indices.
matlab/src/module2_electrostatics/make_module2_batch_parameter_design_2d.m	Legacy wrapper exposing the old parameter-field names for compatibility.
matlab/src/module2_electrostatics/generate_module2_batch_fem_dataset_2d.m	Builds one operator, performs one sparse Cholesky factorization, solves all source vectors, and returns the compact dataset contract without file I/O or plotting.
matlab/src/module2_electrostatics/validate_module2_batch_fem_dataset_2d.m	Checks schema, shapes, finiteness, disjoint complete-case splits, tagged voltages, JJ treatment, residuals, balance, and factorization reuse.
matlab/src/module2_electrostatics/write_module2_batch_fem_summary.m	Writes the dataset manifest, split membership, solver counts, and worst diagnostics.
matlab/src/module2_electrostatics/plot_module2_batch_fem_case_2d.m	Saves one selected charge/potential audit figure; never called by the numerical generator.
matlab/cases/case_module2*.m	Convenience parameter factories for all six built-in cases.
matlab/tests/test_module2*.m	Standalone FEM verification and sanity tests. PINN and CPINN tests with similar prefixes are outside this note's scope.

6 Exact execution sequence

For `main_module2_electrostatics('localized_defect_charge')`, the call sequence is:

1. The driver checks the input and supplies 'localized_defect_charge' when no name is given.
2. `default_module2_params` creates the complete parameter structure and applies the selected case branch.
3. `resolve_module2_mesh_2d` creates the legacy rectangle or validates/reuses the Module 9 geometry.
4. If no explicit charge field is supplied, the built-in synthetic generator creates one nodal ρ field. Otherwise the supplied field is validated and used directly.
5. `assemble_poisson_fem_2d` forms unconstrained K and rhs .
6. `get_module2_dirichlet_nodes` resolves side- or tag-based constrained nodes and rejects conflicting values.
7. `apply_dirichlet_bc` forms Kbc and $rhsbc$.
8. MATLAB solves $\phi = Kbc \setminus rhsbc$.
9. `compute_electric_field_from_potential_2d` computes element and nodal fields.
10. `compute_module2_fem_diagnostics_2d` evaluates the unconstrained free-node residual, exact boundary error, and global reaction balance.
11. The solver returns a **result** structure containing inputs, selected geometry, matrices, solution, fields, boundary provenance, and diagnostics.
12. The driver saves the MAT-file, creates three figures, writes the text summary, and prints a completion report.

This separation is useful while debugging: call the solver directly to isolate numerical behavior, and call the driver when testing the complete input/output workflow.

7 Batch FEM dataset contract and execution sequence

The v2 batch contract is field-to-field:

$$\boxed{\rho^{(m)} \longrightarrow \Phi^{(m)}}. \quad (7.1)$$

The default pilot still creates these input fields from one elliptical Gaussian effective-defect cloud with generator metadata

$$\boldsymbol{\mu}_{\text{gen}} = [C_{\text{peak}}, x_c, z_c, \sigma_x, \sigma_z]. \quad (7.2)$$

The deterministic generator design contains a zero control and 16 charged fields spanning amplitude, depth, lateral displacement, and anisotropic width. Its 11 training, three validation, and three test entries are indices of *complete field configurations*; nodes from a held-out field are never placed in the training set. The five values in $\boldsymbol{\mu}_{\text{gen}}$ are provenance only and are not the canonical neural-surrogate inputs.

The efficient path is:

1. build and validate one Module 9 geometry;
2. resolve the left and right electrode nodes once, with both voltages grounded for this first dataset;
3. assemble the silicon stiffness matrix K once and apply the tagged boundary rows and columns once;
4. compute one sparse Cholesky factor $K_{\text{bc}} = LL^T$;
5. generate or import one complete nodal $\boldsymbol{\rho}$ field for each case, assemble only its source vector, apply the fixed-value right-hand-side correction, and solve $L\mathbf{y} = \mathbf{b}_{\text{bc}}$ followed by $L^T\boldsymbol{\Phi} = \mathbf{y}$;
6. record compact scalar residual, boundary, balance, and range diagnostics;
7. validate the aggregate contract and save one MAT-file.

The saved v2 structure stores **mesh** and **tags** once. The matrices **rho** and **phi** contain one row per mesh node and one column per complete field case; these two matrices are the canonical surrogate input/output pair. The Gaussian generator parameters are stored only under **syntheticGenerator** as provenance metadata. The fields **trainCases**, **validationCases**, and **testCases** identify whole field columns. The stiffness matrix, Cholesky factor, and full residual vectors are intentionally omitted from the saved training dataset.

Linearity decision. The pilot keeps both electrode voltages fixed at zero. For the current linear Poisson equation, nonzero contact-voltage contributions can later be supplied exactly through precomputed Laplace basis solutions and superposition. The first field-to-field surrogate therefore focuses on the charge-field response without wasting capacity on a contribution already known analytically.

8 Parameter reference

All physical inputs are in SI units unless noted otherwise. The default values below are defined before the case-specific switch is evaluated.

8.1 Physical constants and material

Field	Default	Units	Meaning
q	$1.602176634 \times 10^{-19}$	C	Elementary charge magnitude.
eps0	$8.8541878128 \times 10^{-12}$	F/m	Vacuum permittivity.
eps_rel_si	11.7	dimensionless	Relative permittivity of silicon.
eps_si	eps_rel_si*eps0	F/m	Scalar permittivity passed to element assembly. Recompute it after changing either factor.

8.2 Geometry and mesh

Field	Default	Units	Meaning
Lx	10×10^{-6}	m	Domain length in x .
Ly	4×10^{-6}	m	Domain length in y .
nx	41	nodes	Number of structured node locations along x . Must be at least two.
ny	21	nodes	Number of structured node locations along y . Must be at least two.
geometryMode	rectangle	—	Selects rectangle or module9_transmon . The transmon cases set this automatically.

Field	Default	Units	Meaning
module9Geometry	empty	structure	Optional prebuilt geometry. When supplied, Module 2 reuses its node coordinates, connectivity, and tags exactly.
coordinateNames	{x,y}	–	Becomes {x,z} for Module 9 cases.

The node count is $N = n_x n_y$, and the triangle count is

$$N_e = 2(n_x - 1)(n_y - 1). \quad (8.1)$$

Increasing `nx` and `ny` improves spatial resolution but increases matrix size, assembly time, solution time, and output size.

For `module9_transmon`, `nx` and `ny` are synchronized from the accepted nonuniform geometry and the legacy rectangular mesh generator is bypassed. To prove exact shared-mesh reuse:

```

1 shared = build_module9_transmon_geometry_2d();
2 p = default_module2_params('transmon_laplace');
3 p.module9Geometry = shared;
4 r = solve_poisson_defect_space_charge_2d(p);
5 assert(isequal(r.mesh.nodes, shared.mesh.nodes))
6 assert(isequal(r.mesh.elems, shared.mesh.elems))

```

8.3 Canonical charge-field input and synthetic source controls

The preferred direct-solver call supplies an explicit nodal field:

```

1 [mesh,~,~] = resolve_module2_mesh_2d(params);
2 rho = ... % one C/m^3 value per mesh node
3 chargeField = make_module2_charge_field_2d(mesh, rho);
4 result = solve_poisson_defect_space_charge_2d(params, chargeField);

```

A raw numeric nodal `rho` vector is also accepted as the second solver argument. For coupled use, `compose_module2_charge_field_2d` accepts scalar or nodal component fields and applies the semiconductor sign convention before solving.

The scalar fields below remain available because the built-in named cases and Gaussian pilot generator use them. They are synthetic/demo controls, not a restriction on the solver input.

Field	Default	Units	Meaning
n	0	m ⁻³	Uniform electron concentration, entering with a negative sign.
p	0	m ⁻³	Uniform hole concentration, entering with a positive sign.
ND_plus	0	m ⁻³	Uniform ionized donor concentration.
NA_minus	0	m ⁻³	Uniform ionized acceptor concentration.
Cdef_background	0	m ⁻³	Uniform effective defect concentration.
Cdef_peak	2.0×10^{20}	m ⁻³	Peak Gaussian defect concentration before the case switch.
Cdef_x0	$0.5L_x$	m	Gaussian center in x .
Cdef_y0	$0.5L_y$	m	Gaussian center in y .
Cdef_sigma_x	$0.10L_x$	m	Gaussian standard deviation in x . Must be positive.
Cdef_sigma_y	$0.15L_y$	m	Gaussian standard deviation in y . Must be positive.
zdef	1.0	dimensionless	Effective charge number per defect. Positive gives positive defect charge.
rho_uniform	case-specific	C/m ³	When this field exists, it bypasses all carrier, dopant, and defect calculations and supplies an explicit uniform source.

When the synthetic Gaussian generator is selected and **rho_uniform** does not exist, it evaluates

$$C_{\text{def}}(x, y) = C_{\text{bg}} + C_{\text{peak}} \exp \left[-\frac{1}{2} \left(\frac{x - x_0}{\sigma_x} \right)^2 - \frac{1}{2} \left(\frac{y - y_0}{\sigma_y} \right)^2 \right], \quad (8.2)$$

$$\rho(x, y) = q \left[p - n + N_D^+ - N_A^- + z_{\text{def}} C_{\text{def}}(x, y) \right]. \quad (8.3)$$

Synthetic-source pitfall. The fields above control only the built-in synthetic generator. If an explicit `chargeField` or numeric `rho` vector is passed to the solver, that nodal field is authoritative and the Gaussian source controls do not modify it.

8.4 Boundary conditions

Each side has a `type`. A Dirichlet side also has `value`; a nominal Neumann side has `dphidn`. The defaults are grounded left and right contacts with natural top and bottom boundaries:

```
1 params.bc.left.type = 'dirichlet';
2 params.bc.left.value = 0.0;
3 params.bc.right.type = 'dirichlet';
4 params.bc.right.value = 0.0;
5 params.bc.bottom.type = 'neumann';
6 params.bc.bottom.dphidn = 0.0;
7 params.bc.top.type = 'neumann';
8 params.bc.top.dphidn = 0.0;
```

Only the `'dirichlet'` branches are explicitly processed. All other sides contribute the natural homogeneous boundary term of the weak form. A nonzero `dphidn` value is not yet integrated.

For transmon cases, `params.bc.named` points to `electrode.left_electrode` and `electrode.right_electrode`. The collector combines side and named-tag nodes, removes compatible duplicates, and throws an error if the same node receives incompatible voltages. It never applies a boundary condition to `jj_sensitive`.

8.5 Output controls

Field	Default	Effect in the top-level driver
<code>outputDir</code>	<code>outputs/ module2_2d</code>	Resolved relative to the directory containing the top-level driver.
<code>makePlots</code>	<code>true</code>	Calls <code>plot_module2_result_2d</code> .
<code>saveMat</code>	<code>true</code>	Saves the complete <code>result</code> structure.

The text summary is always written by the top-level driver, independent of `makePlots` and `saveMat`.

9 Custom runs without editing the baseline defaults

For exploratory work, create a parameter structure, override fields in the command window or a new script, and call the numerical solver directly. This preserves the validated defaults.

9.1 Mesh-refined localized source

```

1  setup_project_paths
2  params = default_module2_params('localized_defect_charge');
3  params.nx = 81;
4  params.ny = 41;
5  params.Cdef_peak = 5.0e20;
6  params.zdef = -1.0;
7  result = solve_poisson_defect_space_charge_2d(params);
8
9  fprintf('Potential range: [%g, %g] V\n', ...
10         min(result.phi), max(result.phi));

```

Changing `zdef` from `+1` to `-1` reverses the defect-charge contribution. With symmetric grounded contacts and no other charge, the potential perturbation should reverse sign.

9.2 Uniform dopant-only source

```

1  params = default_module2_params('zero_charge');
2  params.ND_plus = 1.0e20;
3  params.NA_minus = 0.0;
4  result = solve_poisson_defect_space_charge_2d(params);

```

Because the concentrations are treated as prescribed uniform values, this is a source-driven electrostatic experiment, not a self-consistent semiconductor equilibrium calculation.

9.3 Saving a direct-solver result deliberately

```

1  outputDir = fullfile(pwd, 'outputs', 'module2_2d', 'custom_probe');
2  if ~exist(outputDir, 'dir')
3      mkdir(outputDir);
4  end
5  save(fullfile(outputDir, 'custom_probe_results.mat'), 'result');
6  plot_module2_result_2d(result, outputDir);
7  write_module2_summary(result, outputDir);

```

Assign a unique `params.caseName` before plotting or summary writing if several custom cases share the same output directory.

10 Returned data structure: what to inspect

The numerical solver returns a structure with the following top-level fields.

Field	Contents
<code>result.params</code>	Complete parameter structure used for the solve.
<code>result.mesh</code>	Nodes, element connectivity, coordinate vectors, dimensions, domain lengths, and four boundary-node sets.
<code>result.geometry</code>	Validated Module 9 geometry, tags, materials, and schema for transmon cases; empty for legacy rectangles.
<code>result.rho</code>	$N \times 1$ nodal charge-density vector.
<code>result.K</code>	Unconstrained sparse stiffness matrix.
<code>result.rhs</code>	Unconstrained assembled source vector.
<code>result.Kbc</code>	Sparse matrix after Dirichlet insertion.
<code>result.rhsbc</code>	Boundary-modified right-hand side.
<code>result.fixedNodes</code>	Constrained global node indices.
<code>result.fixedValues</code>	Prescribed potentials corresponding to <code>fixedNodes</code> .
<code>result.bcInfo</code>	Provenance for each side- or named-tag constraint and the final fixed-node count.
<code>result.phi</code>	Solved nodal potential vector.
<code>result.field</code>	Element and nodal electric-field arrays, magnitudes, centroids, and areas.
<code>result.elementData</code>	Element areas and centroids recorded during assembly.
<code>result.diagnostics</code>	Free-node residual, Dirichlet error, constraint reactions, assembled source, and global balance.
<code>result.maxAbsPhi</code>	Maximum absolute nodal potential.
<code>result.maxAbsE</code>	Maximum area-averaged nodal field magnitude.

Useful mesh subfields include `nodes` ($N \times 2$ coordinates), `elems` ($N_e \times 3$ connectivity), `x`, `y`, `nx`, `ny`, and `boundary.left/right/top/bottom`. Useful field subfields include `Eelem`, `Ex_elem`, `Ey_elem`, `Enodal`, `Ex_nodal`, `Ey_nodal`, `Emag_elem`, and `Emag_nodal`.

11 How to probe a run numerically

11.1 Dimensions, ranges, and finiteness

```

1 size(result.mesh.nodes)
2 size(result.mesh.elems)
3 size(result.Kbc)
4
5 [min(result.rho), max(result.rho)]
6 [min(result.phi), max(result.phi)]
7 [min(result.field.Ex_nodal), max(result.field.Ex_nodal)]
8 [min(result.field.Ey_nodal), max(result.field.Ey_nodal)]
9
10 assert(all(isfinite(result.rho)))
11 assert(all(isfinite(result.phi)))
12 assert(all(isfinite(result.field.Enodal), 'all'))

```

For MATLAB releases that do not accept 'all' in this reduction, replace the final assertion with the equivalent vectorized form:

```

1 assert(all(isfinite(result.field.Enodal(:))))

```

11.2 Linear-system residual

After boundary insertion, the computed solution should satisfy the modified system to floating-point accuracy:

```

1 algebraicResidual = result.Kbc * result.phi - result.rhsbc;
2 relativeResidual = norm(algebraicResidual, 2) / ...
3     max(norm(result.rhsbc, 2), eps);
4 fprintf('Relative algebraic residual = %.3e\n', relativeResidual);

```

This checks whether the linear system was solved. It does *not* by itself prove that the PDE, source, or boundary conditions were assembled correctly.

11.3 Boundary-condition error

```

1 bcError = max(abs(result.phi(result.fixedNodes) - ...
2     result.fixedValues));
3 fprintf('Maximum Dirichlet error = %.3e V\n', bcError);

```

The solver computes this quantity and the two conservation diagnostics automatically:

```
1 result.diagnostics.freeResidualRelative
2 result.diagnostics.dirichletErrorInf
3 result.diagnostics.globalBalanceRelative
4 result.bcInfo.entries
```

The global check sums the Dirichlet constraint reactions and the assembled volume source. With homogeneous natural boundaries, that sum should be zero to solver tolerance.

11.4 Matrix structure and conditioning

```
1 figure; spy(result.Kbc); title('Boundary-modified FEM sparsity');
2 symmetryDefect = norm(result.Kbc - result.Kbc.', 'fro') / ...
3     max(norm(result.Kbc, 'fro'), eps);
4 conditionEstimate = condest(result.Kbc);
```

The unconstrained electrostatic stiffness matrix is symmetric. The boundary routine zeros both constrained rows and columns, so Kbc should also be symmetric to roundoff. A very large condition estimate can accompany poor scaling, extreme geometry aspect ratio, or excessive mesh refinement.

11.5 Probe the node nearest a physical coordinate

```
1 target = [0.50*result.params.Lx, 0.50*result.params.Ly];
2 distance2 = sum((result.mesh.nodes - target).^2, 2);
3 [~, nodeID] = min(distance2);
4
5 fprintf('Node %d at (%g,%g) m\n', nodeID, ...
6     result.mesh.nodes(nodeID,1), result.mesh.nodes(nodeID,2));
7 fprintf('rho = %g C/m^3, phi = %g V\n', ...
8     result.rho(nodeID), result.phi(nodeID));
9 fprintf('E = (%g,%g) V/m\n', ...
10     result.field.Ex_nodal(nodeID), ...
11     result.field.Ey_nodal(nodeID));
```

11.6 Inspect one triangle

```
1 elementID = 1;
2 localNodes = result.mesh.elems(elementID,:);
3 coordinates = result.mesh.nodes(localNodes,:);
4 [Ke, area, gradN] = compute_element_stiffness_triangle( ...
```

```

5     coordinates, result.params.eps_si);
6 localRho = result.rho(localNodes);
7 fe = compute_element_source_triangle(area, localRho);

```

This is the most direct way to study the connection between triangle geometry, basis gradients, element matrix, source values, and global assembly.

12 Generated outputs and how to check them

For a case named `<case>`, the default driver writes:

- `<case>_results.mat`: complete `result` structure;
- `<case>_potential.png`: nodal potential on the triangular mesh;
- `<case>_space_charge.png`: nodal space-charge density;
- `<case>_electric_field_magnitude.png`: area-averaged nodal field magnitude;
- `<case>_summary.txt`: scalar run summary.

Legacy outputs use `matlab/outputs/module2_2d/`; the two transmon cases use `matlab/outputs/module2_transmon_2d/`. Transmon figures use millimetres, label the vertical coordinate z , and overlay the two electrode groups plus the JJ-sensitive interval. After a run, check:

1. the summary reports the requested domain and mesh;
2. the number of nodes is `nx*ny`;
3. the number of triangles is `2*(nx-1)*(ny-1)`;
4. charge, potential, and field ranges are finite;
5. the potential agrees with the prescribed left and right values;
6. plot symmetry or sign agrees with the selected source;
7. the MAT-file loads and reproduces the scalar values in the summary.

```

1 loaded = load(fullfile('outputs', 'module2_2d', ...
2     'localized_defect_charge_results.mat'));
3 loaded.result.maxAbsE

```

The batch driver uses `matlab/outputs/module2_batch_fem_2d/` and writes:

- `module2_transmon_field_to_field_gaussian_pilot_v2.mat`: current complete field-to-field dataset structure;
- `module2_transmon_field_to_field_gaussian_pilot_v2_summary.txt`: current case manifest, split membership, operator reuse counts, and worst diagnostics;
- older `module2_transmon_gaussian_batch_v1.*` files may remain as historical pre-v2 outputs;
- `<case>_batch_fem_audit.png`: only when representative plotting is explicitly enabled.

Load and inspect the aggregate contract with:

```
1 loaded = load(fullfile('outputs', 'module2_batch_fem_2d', ...
2     'module2_transmon_field_to_field_gaussian_pilot_v2.mat'));
3 size(loaded.dataset.rho)           % expected: nNodes by 17
4 size(loaded.dataset.phi)           % expected: nNodes by 17
5 loaded.dataset.syntheticGenerator.parameters % provenance only: 17 by 5
6 loaded.dataset.trainCases
7 loaded.dataset.validationCases
8 loaded.dataset.testCases
```

13 Verification tests

13.1 Run the FEM-only test suite

From matlab/:

```
1 setup_project_paths
2 test_module2_fem_all_2d
```

This master runner executes the legacy analytical/sanity tests, the new explicit-field and component-field contract tests, the focused Module 2-on-Module 9 integration tests, and the batch-dataset suite. For a focused diagnosis, use:

- `test_module2_transmon_integration_2d` for the geometry-aware single-case path;
- `test_module2_batch_fem_dataset_2d` for factorized dataset generation.

The following similarly named tests validate different code paths and are outside this FEM test group:

- `test_module2_pinn_electrostatics`;

- `test_module2_cpinn_electrostatics`.

Those tests can require the Deep Learning Toolbox.

To stop on the first failure while running the FEM group programmatically:

```

1 femTests = { ...
2     @test_module2_zero_charge_2d, ...
3     @test_module2_linear_potential_2d, ...
4     @test_module2_uniform_space_charge_2d, ...
5     @test_module2_localized_defect_charge_2d, ...
6     @test_module2_explicit_charge_field_2d, ...
7     @test_module2_charge_component_fields_2d, ...
8     @test_module2_transmon_tagged_bc_2d, ...
9     @test_module2_transmon_trapped_charge_2d, ...
10    @test_module2_transmon_mesh_reuse_2d, ...
11    @test_module2_transmon_boundary_guards_2d, ...
12    @test_module2_batch_fem_dataset_2d};
13
14 for k = 1:numel(femTests)
15     feval(femTests{k});
16 end

```

13.2 What each test proves

Test	Pass criterion	Interpretation
<code>test_module2_zero_charge_2d</code>	$\max \phi < 10^{-12} \text{ V}$	Zero source and zero contact potential produce the zero solution.
<code>test_module2_linear_potential_2d</code>	Maximum nodal error below 10^{-10} V	Linear T3 basis reproduces the exact charge-free linear solution.
<code>test_module2_uniform_space_charge_2d</code>	Relative maximum potential error below 5×10^{-3}	Source assembly approaches the analytical one-dimensional quadratic solution on the default mesh.
<code>test_module2_localized_defect_charge_2d</code>	Finite potential, positive maximum source, and positive maximum potential	Localized positive charge produces a physically plausible response. This is a sanity test, not a convergence proof.

Test	Pass criterion	Interpretation
test_module2_explicit_charge_field_2d	Arbitrary non-Gaussian nodal ρ is preserved exactly and produces a converged solve	The canonical solver path no longer requires Gaussian parameters.
test_module2_charge_component_fields_2d	Spatial/scalar component fields reproduce the signed semiconductor charge formula exactly	$p, n, N_D^+, N_A^-, C_{\text{def}}, z_{\text{def}}$ can be assembled into total nodal ρ .
test_module2_transmon_tagged_bc_2d	Exact 0/1 mV tag voltages, disjoint electrodes, bounded potential, residual and balance below 10^{-9} , and three PNG smoke outputs	Module 2 consumes the named electrode tags without shorting through the JJ and all geometry-aware plot paths execute.
test_module2_transmon_trapped_charge_2d	Positive finite response, exact grounded tags, residual and balance below 10^{-9}	Localized defect charge is evaluated on the Module 9 mesh and drives the expected sign.
test_module2_transmon_mesh_reuse_2d	Node coordinates, connectivity, and electrode edge IDs match exactly	A prebuilt shared Module 9 object is passed through without remeshing or retagging.
test_module2_transmon_boundary_guards_2d	Unknown tag and conflicting-voltage error identifiers are caught	Misspelled tags cannot fall back silently, and one node cannot receive incompatible constraints.
test_module2_batch_dataset_contract_2d	All aggregate validation checks pass	The v2 dataset stores mandatory nodal ρ input and nodal ϕ output fields, complete-field splits, tags, diagnostics, and Gaussian provenance only under <code>syntheticGenerator</code> .

Test	Pass criterion	Interpretation
<code>test_module2_batch_factorization_reuse_2d</code>	One stiffness assembly, one factorization, and one solve per case	The batch path reuses the operator rather than repeating the expensive setup work.
<code>test_module2_batch_single_case_equivalence_2d</code>	Potential agrees with the validated single-case path to relative tolerance 10^{-11}	Factorized reuse changes execution strategy without changing the electrostatic solution.
<code>test_module2_batch_split_integrity_2d</code>	Splits are disjoint and cover every case once	No complete-field leakage is introduced before field-surrogate training.
<code>test_module2_batch_reproducible_design_2d</code>	Repeated generator-design construction is identical	The 17-field Gaussian pilot generator is deterministic and auditable.

Every test calls `setup_project_paths`, disables plots and MAT-file saving, calls the numerical solver directly, and uses MATLAB `assert` statements. A printed `passed` line means all assertions in that function completed.

13.3 Batch-test command

```
matlab -batch "cd('C:/path/to/RadiationEffects_proj2/matlab'); setup_project_paths;
test_module2_fem_all_2d;"
```

13.4 Mesh-refinement check

Passing one fixed-mesh test is necessary but not sufficient for a research calculation. For a convergence study, repeat the same physical case on successively finer meshes and compare a scalar or field quantity. The uniform-charge case has an analytical nodal reference:

```
1 meshSizes = [21, 41, 81, 161];
2 errors = zeros(size(meshSizes));
3
4 for k = 1:numel(meshSizes)
5     params = default_module2_params('uniform_space_charge');
6     params.nx = meshSizes(k);
7     params.ny = 1 + (meshSizes(k)-1)/2;
8     r = solve_poisson_defect_space_charge_2d(params);
```

```

9
10     x = r.mesh.nodes(:,1);
11     exact = (params.rho_uniform/(2*params.eps_si)) .* ...
12         x .* (params.Lx-x);
13     errors(k) = max(abs(r.phi-exact)) / max(max(abs(exact)), eps);
14 end
15
16 loglog(meshSizes, errors, 'o-');
17 xlabel('Number of x nodes'); ylabel('Relative max error'); grid on;

```

For this special problem, the exact solution is quadratic in x , whereas the T3 approximation is piecewise linear. Error should decrease with refinement. A refinement study for a coupled, highly localized source should additionally monitor peak potential, field magnitude, integrated charge, and spatial profiles.

14 Debugging and interactive code tracing

14.1 Stop at the first MATLAB error

```

1 dbstop if error
2 result = main_module2_electrostatics('localized_defect_charge');

```

MATLAB will pause at the line that raised the error. Use the Workspace, `whos`, and the editor's step controls to inspect local variables. Clear the diagnostic stop later with `dbclear if error`.

14.2 Enter the solver at its first line

```

1 dbstop in solve_poisson_defect_space_charge_2d at 1
2 result = main_module2_electrostatics('localized_defect_charge');

```

Step through in this order: `mesh`, `rho`, `K/rhs`, `constrained nodes`, `Kbc/rhsbc`, `phi`, then `field`. This sequence mirrors the actual numerical dependency chain.

14.3 Inspect source code and resolution

```

1 open main_module2_electrostatics
2 open solve_poisson_defect_space_charge_2d
3 open assemble_poisson_fem_2d
4 which apply_dirichlet_bc -all

```

If edits are not reflected in execution, check `which -all`, then use `clear functions` and `rehash path`. Do not continue until MATLAB resolves the intended project copy.

15 Common failure modes and diagnostic actions

Symptom	Likely cause	Action
Undefined function for a Module 2 file	Project paths were not added, or MATLAB is in the wrong extracted copy.	Run <code>setup_project_paths;</code> in spect <code>which ... -all</code> .
Unknown Module 2 case name	The driver accepts only the six strings in the default-parameter switch.	Use an exact built-in name or call the solver with an overridden parameter structure.
Triangle has non-positive area	Connectivity was changed to clockwise order, or the mesh contains a degenerate element.	Inspect the reported element coordinates and restore counter-clockwise connectivity.
Singular or nearly singular matrix warning	No Dirichlet reference potential, disconnected mesh, or invalid constraints.	Confirm <code>fixedNodes</code> is nonempty and the mesh is connected.
Changing <code>dphidn</code> has no effect	Nonzero Neumann boundary integration is not implemented.	Keep natural flux zero or extend assembly with the correct boundary integral.
Changing defect parameters has no effect	<code>rho_uniform</code> exists and bypasses the general source calculation.	Remove the field or start from a non-uniform-source case.
NaN or Inf in charge	Zero/invalid Gaussian width or nonfinite input concentration.	Require positive <code>Cdef_sigma_x/y</code> and inspect every source field.
Unexpected potential sign	Charge sign, effective defect charge, or field convention was misread.	Inspect <code>rho</code> ; remember positive source with grounded contacts gives a positive potential, while $\mathbf{E} = -\nabla\phi$.
Unexpected corner potential	Two Dirichlet sources prescribe different values at one node.	Make the values compatible; the collector now rejects conflicting constraints explicitly.

Symptom	Likely cause	Action
No output files after a direct solver call	Saving, plotting, and summary writing are driver responsibilities.	Call the three output functions explicitly or use the top-level driver.
Plot creation fails in a headless environment	Graphics renderer is unavailable even for invisible figures.	Use the direct solver route and omit <code>plot_module2_result_2d</code> ; inspect numerical arrays and summaries.
Excessive memory or long solve time	<code>nx</code> and <code>ny</code> create too many nodes/elements for the environment.	Start with the defaults, refine gradually, and monitor <code>whos result</code> .
Unphysical magnitude	A concentration or length was entered in centimeters, micrometers, or cubic centimeters without SI conversion.	Convert lengths to meters and concentrations to m^{-3} before solving.

16 Adding a new standalone case safely

There are two useful levels of customization.

16.1 One-off experiment

Do not edit the parameter factory. Create and override a structure in a script, call the solver, inspect the result, and save it under a unique name. This is the recommended exploratory path because it cannot silently alter the six built-in cases.

16.2 New named regression or demonstration case

For a permanent named case:

1. Add a new `case` branch in `default_module2_params.m`.
2. Set every field that differs from the common defaults.
3. Optionally add a convenience wrapper in `matlab/cases/`.
4. Add a test in `matlab/tests/` with a quantitative pass criterion.

5. Run `test_module2_fem_all_2d` to detect legacy or transmon regressions.
6. Run the new case through the top-level driver and inspect every output.
7. Document the case name, units, reference solution or expected trend, and limitations.

Adding only a wrapper in `matlab/cases/` does not make the name acceptable to the current top-level driver; the switch in `default_module2_params.m` remains authoritative.

17 Preparing for Module 3 and Module 5 coupling

The field interface required for coupling is now implemented. Upstream modules may provide one or more nodal concentration fields and Module 2 may compose them as

$$\boldsymbol{\rho} = q \left(\boldsymbol{p} - \boldsymbol{n} + \boldsymbol{N}_D^+ - \boldsymbol{N}_A^- + \sum_i \boldsymbol{z}_i \odot \boldsymbol{C}_i \right). \quad (17.1)$$

The helper `compose_module2_charge_field_2d.m` already accepts scalar or nodal values for the current reduced set $p, n, N_D^+, N_A^-, C_{\text{def}}, z_{\text{def}}$. The resulting `module2_charge_field_v1` structure can be passed directly to the FEM solver.

Before connecting live Module 3 or Module 5 outputs, still define and test:

- whether the modules share exactly the same node ordering and mesh;
- the interpolation/conservative-projection rule when their meshes differ;
- concentration units and effective charge signs;
- species ordering and metadata;
- conservation or interpolation error checks;
- behavior at times when one component field is unavailable;
- whether an upstream module supplies components separately or an already combined total-charge map.

The key architectural change is that no Gaussian fit is required between modules. A physically generated spatial field can be passed through the same validated $\boldsymbol{\rho}$ contract used by synthetic tests.

18 Recommended verification ladder before using a result

Use the following order when changing the FEM code:

1. **Path check:** verify that MATLAB resolves the intended files.
2. **Zero solution:** run `test_module2_zero_charge_2d`.
3. **Boundary and sign check:** run `test_module2_linear_potential_2d`.
4. **Source integration check:** run `test_module2_uniform_space_charge_2d`.
5. **Localized-source sanity:** run `test_module2_localized_defect_charge_2d`.
6. **Field contract:** run the two new field-interface tests.
`test_module2_explicit_charge_field_2d`
`test_module2_charge_component_fields_2d`.
7. **Named-tag integration:** run `test_module2_transmon_integration_2d`.
8. **Algebraic checks:** inspect residual, symmetry, fixed-node error, global reaction balance, and finiteness.
9. **Mesh refinement:** establish convergence of the reported quantity.
10. **Physical scale check:** compare charge, potential, and field magnitudes with an independent estimate.
11. **Coupling check:** when importing fields, verify units, interpolation, sign, and total-charge consistency.

Interpretation rule. A successful sparse solve demonstrates that MATLAB found a vector satisfying the assembled algebraic equations. Analytical tests, convergence studies, dimensional checks, and coupling consistency are what establish that those equations represent the intended physical problem.

19 Minimal handoff checklist

Before sending or presenting a Module 2 FEM result, record:

- repository/archive version and MATLAB version;
- case name and any parameter overrides;
- geometry mode and, for transmon cases, Module 9 schema;
- L_x , L_y , $\mathbf{nx}$, and $\mathbf{ny}$;
- permittivity and every nonzero charge contribution;

- boundary types, named tag sources, and values;
- passed tests and mesh-refinement evidence;
- algebraic residual, fixed-boundary error, and global reaction balance;
- output filenames and the exact quantity plotted;
- for batch data, field-dataset schema, complete-field split indices, synthetic-generator provenance when applicable, operator/factorization counts, and worst per-case diagnostics;
- known limitations, especially prescribed sources and absent nonzero-Neumann support.

20 Next development gate

After the complete FEM suite passes, generate the 17-case pilot and inspect its text validation summary:

```
1 [dataset, report] = main_module2_batch_fem_dataset();
```

The next coding gate is to replace the executable pointwise Module 2 PINN with the field-to-field architecture defined in the revised physics note. The neural model should consume the saved nodal ρ field and predict nodal Φ on the accepted geometry. For physics-informed training, the preferred baseline is the differentiable discrete FEM residual $K_{FF}\Phi_{\theta,F} + K_{FD}\Phi_D - \mathbf{b}_F(\rho)$, with complete-field train/validation/test splits and the normalized/adaptive loss weighting described in the physics note. The current `module2_pinn_implementation_note.pdf` remains a guide to the legacy pointwise prototype until that code migration is completed. Do not scale to hundreds of fields until the v2 field contract and a small held-out end-to-end neural pilot pass.